

PROGRAMMATION FONCTIONNELLE 2 : INTERROGATION N°1
avril 2026 — Pierre Rousselin

Exercice 1 : Arbres binaires (non stricts)

On rappelle la définition des arbres binaires (non stricts).

```
Inductive btree (A : Type) :=
| emptyTree
| node (left : btree A) (key : A) (right : btree A).
```

L'argument A est implicite pour les deux constructeurs :

```
Arguments emptyTree {A}.
Arguments node {A}.
```

On se donne l'exemple suivant :

```
Definition plop :=
node
  (node
    emptyTree
    12
    (node emptyTree 10 emptyTree))
  42
  (node
    (node emptyTree 2 emptyTree)
    1
    (node emptyTree 3 emptyTree)).
```

1. (1 point) Représenter graphiquement l'arbre plop.
2. (2 points) Définir la fonction Rocq numLeft qui donne le nombre de nœuds à gauche de leurs parents dans un arbre (la racine n'est pas comptée). Par exemple, on devrait avoir numLeft plop = 2.
3. (1 point) Calculer pas à pas numLeft plop.

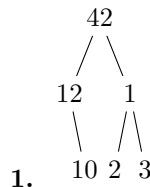
On rappelle la définition de la fonction mirror vue en TD :

```
Fixpoint mirror {A : Type} (t : btree A) :=
match t with
| emptyTree => emptyTree
| node l k r => node (mirror r) k (mirror l)
end.
```

4. (4 points) Donner la preuve papier, en détaillant chaque étape de calcul du lemme suivant :
Lemma mirror_involutive {A : Type} (t : btree t) : mirror (mirror t) = t.
5. (3 points) En donner maintenant la preuve en Rocq.

--- * ---

Correction de l'exercice 1 :



2. **Fixpoint** numLeft {A : Type} (t : btree A) :=
 match t with
 | emptyTree => 0 (* nLeT *)
 | (node emptyTree _ r) => numLeft r (* nLeTl *)
 | (node l _ r) => S (numLeft l + numLeft r) (* nLneTl *)
 end.

ou bien sans sucre :

Fixpoint numLeft' {A : Type} (t : btree A) :=
 match t with
 | emptyTree => 0
 | (node l _ r) => match l with
 | emptyTree => numLeft' r
 | (node _ _ _) => S (numLeft' l + numLeft' r)
 end
 end.

3. On pouvait utiliser des abréviations, raccourcir en utilisant plusieurs règles à la fois, entourer, utiliser des couleurs pour montrer ce qui est transformé en quoi, voire utiliser des dessins pour les sous-arbres... l'essentiel est qu'on comprenne bien les différentes étapes de calcul. Ici, tous les détails, juste pour le plaisir masochiste de mettre vaguement ça en forme en $\text{\LaTeX}$.

$$\begin{aligned}
 \text{numLeft plop} &\stackrel{(\text{nLneTl})}{=} S \left(\text{numLeft}(\text{node emptyTree 12 (node emptyTree 10 emptyTree)}) \right. \\
 &\quad \left. + \text{numLeft} (\text{node (node emptyTree 2 emptyTree) 1 (node emptyTree 3 emptyTree)}) \right) \\
 &\stackrel{(\text{nLeTl})}{=} S \left(\text{numLeft} (\text{node emptyTree 10 emptyTree}) \right. \\
 &\quad \left. + \text{numLeft} (\text{node (node emptyTree 2 emptyTree) 1 (node emptyTree 3 emptyTree)}) \right) \\
 &\stackrel{(\text{nLeTl})}{=} S \left(\text{numLeft emptyTree} \right. \\
 &\quad \left. + \text{numLeft} (\text{node (node emptyTree 2 emptyTree) 1 (node emptyTree 3 emptyTree)}) \right) \\
 &\stackrel{(\text{nLeT})}{=} S \left(0 + \text{numLeft} (\text{node (node emptyTree 2 emptyTree) 1 (node emptyTree 3 emptyTree)}) \right) \\
 &\stackrel{(\text{nLneTl})}{=} S \left(0 + S (\text{numLeft} (\text{node emptyTree 2 emptyTree}) + \text{numLeft} (\text{node emptyTree 3 emptyTree})) \right) \\
 &\stackrel{(\text{nLeTl } 2 \text{ fois})}{=} S \left(0 + S (\text{numLeft emptyTree} + \text{numLeft emptyTree}) \right) \\
 &\stackrel{(\text{nLeT } 2 \text{ fois})}{=} S \left(0 + S (0 + 0) \right) = 2.
 \end{aligned}$$

4. Preuve par induction sur t.

— Dans le cas où $t = \text{emptyTree}$, on a :

$$\text{mirror} (\text{mirror emptyTree}) \stackrel{(\text{mireT})}{=} \text{mirror emptyTree} \stackrel{(\text{mireT})}{=} \text{emptyTree}$$

— On suppose que $t = \text{node } l \ k \ r$, où l et r satisfont :

$$\text{mirror} (\text{mirror } l) = l \quad (\text{IHl})$$

$$\text{mirror} (\text{mirror } r) = r \quad (\text{IHr})$$

Alors,

$$\begin{aligned}
 \text{mirror} (\text{mirror} (\text{node } l \ k \ r)) &\stackrel{(\text{mirnode})}{=} \text{mirror} (\text{node} (\text{mirror } l) \ k \ (\text{mirror } r)) \\
 &\stackrel{(\text{mirnode})}{=} \text{node} (\text{mirror}(\text{mirror } l)) \ k \ (\text{mirror} (\text{mirror } r)) \\
 &\stackrel{(\text{IHl}, \text{IHr})}{=} \text{node } l \ k \ r = t.
 \end{aligned}$$

```

— Lemma mirror_involutive {A : Type} (t : btree t) :
  mirror (mirror t) = t.
Proof.
  induction t as [| l IHl k r IHR].
  - reflexivity.
  - simpl. (* But : node (mirror (mirror l)) k (mirror (mirror r)) *)
    rewrite IHl IHR. reflexivity.
Qed.

```

--- * ---

Exercice 2 : Entiers en binaire

On rappelle la définition du type `positive` qui modélise les entiers naturels non nuls (strictement positifs) écrits en binaire, et les notations suivantes :

```

Inductive positive : Set :=
| xI (p : positive) | x0 (p : positive) | xH.
Notation "1" := xH : positive_scope.
Notation "p ~ 1" := (xI p) : positive_scope.
Notation "p ~ 0" := (x0 p) : positive_scope.

```

- (1 point) Donner la valeur en décimal de l'entier $1 \sim 0 \sim 1 \sim 1 \sim 0$.
- (1 point) Le représenter sans notation, avec uniquement les constructeurs.
- (2 points) Donner la preuve en Rocq du lemme suivant :

```

Lemma foo (p : positive) : p ~ 0 <> p ~ 1.

```

On rappelle la définition des fonctions `succ` et `add` vues en cours :

```

Fixpoint succ (p : positive) :=
  match p with
  | xI q => x0 (succ q)
  | x0 q => xI q
  | xH => x0 xH
  end.
Fixpoint add (p q : positive) :=
  match p with
  | xI p' => match q with
    | xH => x0 (succ p')
    | x0 q' => xI (add p' q')
    | xI q' => x0 (succ (add p' q'))
    end
  | xH => succ q
  | x0 p' => match q with
    | xH => xI p'
    | x0 q' => x0 (add p' q')
    | xI q' => xI (add p' q')
    end
  end.
Notation "p + q" := (add p q).

```

- (1 point) Évaluer en détaillant chaque étape $1 \sim 0 + 1 \sim 0 \sim 1$.
 - (4 points) Écrire la preuve papier détaillée du lemme suivant (en détaillant les étapes de calcul) :
- ```

Lemma add_diag (p : positive) : p + p = p ~ 0.

```

--- \* ---

## Correction de l'exercice 2 :

1.  $16 + 4 + 2 = 22$ .
2.  $\text{xO } (\text{xI } (\text{xI } (\text{xO } \text{xH})))$
3. **intros** H. **discriminate** H.
4.  $1 \sim 0 + 1 \sim 0 \sim 1 = (1 + 1 \sim 0) \sim 1 = (\text{succ } 1 \sim 0) \sim 1 = 1 \sim 1 \sim 1$
5. Par induction sur  $p$ .

— Si  $p = \text{xI } 1$ , alors

$$1 + 1 = \text{xI } + \text{xI } = \text{succ } \text{xI } = \text{xO } \text{xI } = 1 \sim 0.$$

— Si  $p = \text{xO } q$ , où  $q$  satisfait :

$$q + q = q \sim 0, \tag{IHq}$$

alors

$$\text{xO } q + \text{xO } q = \text{xO } (q + q) \stackrel{(\text{IHq})}{=} \text{xO } (q \sim 0) = q \sim 0 \sim 0 = p \sim 0.$$

— Si  $p = \text{xI } q$ , où  $q$  satisfait :

$$q + q = q \sim 0, \tag{IHq}$$

alors

$$\text{xI } q + \text{xI } q = \text{xO } (\text{succ } (q + q)) \stackrel{(\text{IHq})}{=} \text{xO } (\text{succ}(q \sim 0)) = q \sim 1 \sim 0 = p \sim 0.$$

--- \* ---